

Lab No 8:

Programs using CUDA and CUDA-synchronization

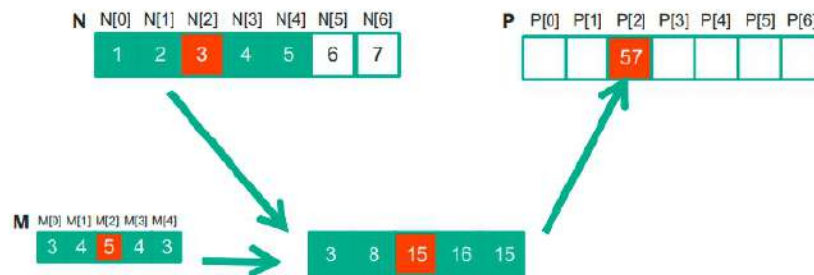
Objectives:

In this lab, students will be able to

1. Learn the concepts to perform 1D convolution
2. Learn to compute time of kernel execution
3. Learn about atomic functions

1D Convolution:

- Convolution is a popular array operation that is used in various forms in signal processing, digital recording, image processing, video processing, and computer vision.
- Convolution is often performed as a filter that transforms signals and pixels into more desirable values. For example, Gaussian filters are convolution filters that can be used to sharpen boundaries and edges of objects in images.
- Mathematically, convolution is an array operation where each output data element is a weighted sum of a collection of neighboring input elements.
- The weights used in the weighted sum calculation are defined by an input mask array, commonly referred to as the convolution mask OR convolution kernel.
- The same convolution mask is typically used for all elements of the array.



- The fact that we use a five-element mask M means that each P element is generated by a weighted sum of the corresponding N element, up to two elements to the left and up to two elements to the right.
- Each weight value is multiplied to the corresponding N element values before the products are summed together.
- In general, the size of the mask tends to be an odd number, which makes the weighted sum calculation symmetric around the element being calculated.

```

__global__ void convolution_1D_basic_kernel(float *N, float *M, float *P,
int Mask_Width, int Width) {

    int i = blockIdx.x*blockDim.x + threadIdx.x;

    float Pvalue = 0;
    int N_start_point = i - (Mask_Width/2);
    for (int j = 0; j < Mask_Width; j++) {
        if (N_start_point + j >= 0 && N_start_point + j < Width) {
            Pvalue += N[N_start_point + j]*M[j];
        }
    }
    P[i] = Pvalue;
}

```

Arithmetic functions:

In a multithreaded scenario, the issue of data inconsistency will arise, if multiple threads modify a single shared memory variable. To overcome this atomic functions need to be used. List of atomic functions, their syntax and explanation is provided below.

atomicAdd():

```
int atomicAdd (int* address, int val);
```

```
unsigned int atomicAdd(unsigned int* address, unsigned int val);
```

```
float atomicAdd(float* address, float val);
```

```
double atomicAdd(double* address, double val);
```

Reads the 16-bit, 32-bit or 64-bit word old located at the address address in global or shared memory, computes (old + val), and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

atomicSub():

```
int atomicSub(int* address, int val);
```

```
unsigned int atomicSub(unsigned int* address, unsigned int val);
```

Reads the 32-bit word old located at the address address in global or shared memory, computes (old - val), and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

atomicExch():

```
int atomicExch(int* address, int val);  
  
unsigned int atomicExch(unsigned int* address, unsigned int val);  
  
float atomicExch(float* address, float val);
```

Reads the 32-bit word old located at the address address in global or shared memory and stores val back to memory at the same address. These two operations are performed in one atomic transaction. The function returns old.

atomicMin():

```
int atomicMin(int* address, int val);  
  
unsigned int atomicMin(unsigned int* address, unsigned int val);
```

Reads the 32-bit word old located at the address address in global or shared memory, computes the minimum of old and val, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

atomicMax():

```
int atomicMax(int* address, int val);  
  
unsigned int atomicMax(unsigned int* address, unsigned int val);
```

Reads the 32-bit word old located at the address address in global or shared memory, computes the maximum of old and val, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

atomicInc():

```
unsigned int atomicInc(unsigned int* address, unsigned int val);
```

Reads the 32-bit word old located at the address address in global or shared memory, computes ((old >= val) ? 0 : (old+1)), and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

atomicDec():

```
unsigned int atomicDec(unsigned int* address, unsigned int val);
```

Reads the 32-bit word old located at the address address in global or shared memory, computes $((old == 0) \mid (old > val)) ? val : (old - 1)$, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

Solved Example:

A CUDA program which takes a string as input and determines the number of occurrences of a character 'a' in the string. This program uses atomicAdd() function.

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <conio.h>
#define N 1024

__global__ void CUDACount(char* A, unsigned int *d_count){
    int i = threadIdx.x;
    if(A[i]=='a')
        atomicAdd(d_count,1);
}

int main() {
    char A[N];
    char *d_A;
    unsigned int *count=0,*d_count,*result;
    printf("Enter a string");
    gets(A);
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    cudaEventRecord(start, 0);
    cudaMalloc((void**)&d_A, strlen(A)*sizeof(char));
    cudaMalloc((void **)&d_count,sizeof(unsigned int));
    cudaMemcpy(d_A, A, strlen(A)*sizeof(char), cudaMemcpyHostToDevice);
    cudaMemcpy(d_count,count,sizeof(unsigned int),cudaMemcpyHostToDevice);
```

```

cudaError_t error = cudaGetLastError();
if (error != cudaSuccess) {
    printf("CUDA Error1: %s\n", cudaGetErrorString(error));
}
    CUDACount<<<1,strlen(A)>>>(d_A,d_count);
error = cudaGetLastError();
if (error != cudaSuccess) {
    printf("CUDA Error2: %s\n", cudaGetErrorString(error));
}
cudaEventRecord(stop, 0);
cudaEventSynchronize(stop);
float elapsedTime;
cudaEventElapsedTime(&elapsedTime, start, stop);
    cudaMemcpy(result, d_count, sizeof(unsigned int), cudaMemcpyDeviceToHost);
    printf("Total occurrences of a=%u",result);
printf("Time Taken=%f",elapsedTime);
    cudaFree(d_A);
    cudaFree(d_count);
    printf("\n");
    getch();
    return 0;
}

```

Explanation:

The kernel uses `atomicAdd` function with 1 as the value each time a character 'a' occurs in the string. The instructions given in bold are present to find the time. As in OpenCL you need to declare an event, register the event and record the time before kernel execution and after kernel execution. You must synchronize the event so that main thread can capture the time of execution of kernel. After that **`cudaEventElapsedTime(&elapsedTime, start, stop)`** will give the difference between the recorded stop and start time and store the value in the variable `elapsedTime` which is of type float. A negative time value means there is something wrong in the CUDA code. To find out you use the code given in bold and italics. It will display the error message present in CUDA code. Call it once before calling the kernel and once after calling kernel. If first call throws an error message then error is present in CUDA API which precedes the kernel. If second call throws the error message then error

is present in the kernel code.

Lab Exercise:

- 1) Implement a program in CUDA which performs convolution operation on one dimensional input array N of size width using a mask array M of size mask_width to produce the resultant one-dimensional array P of size width.
- 2) Implement a program in CUDA to perform odd even transposition sort in parallel.
- 3) Implement a program in CUDA to perform parallel rank sort in parallel.
- 4) Implement a program in CUDA to count the number of times a given word is repeated in a sentence. (Use Atomic function)
- 5) Implement a CUDA program that reads a string and replaces every vowel with '*'. Use appropriate atomic function to replace the character with *.

Input String S: Parallel CUDA

Output String RS: P*r*ll*I C*D*

Additional Exercise:

- 1) Implement a CUDA program that reads a string S and produces the string **RS** as follows: Input string **S**: PCAP Output string **RS**: PCAPPCAPCP
Note: Each work item copies required number of characters from S in RS.
- 2) Implement a CUDA program which reads a string consisting of N words and reverse each word of it in parallel.
- 3) Implement a CUDA program that takes a string Sin as input and one integer value N and produces an output string , Sout, in parallel by concatenating input string Sin, N times as shown below.
Input: Sin = "Hello" N = 3
Output: Sout = "HelloHelloHello"
Note: Every thread copies the same character from the Input string S, N times to the re- quired position.